
EGCI427 Lab — RESTful API Design with MongoDB & Mongoose

Learning Goals

- Install and connect to MongoDB (local or Atlas)
- Define Mongoose schemas and models
- Build a full CRUD REST API backed by MongoDB
- Understand validation and query filters

Part A — Beginner (0:00–0:40): MongoDB Setup & Connection

Step 1: Choose ONE option

Option 1 — MongoDB Atlas (cloud, recommended, no install needed)

1. Sign up free at https://www.mongodb.com/cloud/atlas/register
2. Create a free (M0) cluster.
3. Under **Database Access**, create a user with a password.
4. Under **Network Access**, add 0.0.0.0/0 (allow from anywhere, for class use).
5. Click **Connect** → **Drivers** → copy the connection string, e.g.:
   mongodb+srv://<username>:<password>@cluster0.xxxxx.mongodb.net/egci427

Option 2 — Local MongoDB (Ubuntu/Debian)

```
sudo apt-get install -y mongodb
sudo systemctl start mongodb
```

Local connection string: mongodb://localhost:27017/egci427

Step 2: Create the project

```
mkdir egci427-week4 && cd egci427-week4
npm init -y
npm install express mongoose dotenv
npm install --save-dev nodemon
```

Add to package.json:

```
"type": "module",
"scripts": {
```

```
  "dev": "nodemon server.js"
}
```

Step 3: Create .env

```
echo "PORT=3000
MONGO_URI=mongodb+srv://<username>:<password>@cluster0.xxxxx.mongodb.net/egci427" >
.env
echo "node_modules/
.env" > .gitignore
```

(Replace with your local URI if using Option 2.)

Step 4: Create server.js and connect to MongoDB

```
// server.js
import express from 'express';
import mongoose from 'mongoose';
import dotenv from 'dotenv';

dotenv.config();
const app = express();
const PORT = process.env.PORT || 3000;

app.use(express.json());

mongoose
  .connect(process.env.MONGO_URI)
  .then(() => console.log('✅ MongoDB connected'))
  .catch((err) => console.error('❌ MongoDB connection error:', err.message));

app.get('/', (req, res) => res.send('EGCI427 API with MongoDB'));

app.listen(PORT, () => console.log(`Server running at http://localhost:${PORT}`));
```

Step 5: Run it

npm run dev

You should see ✅ MongoDB connected.

✅ **Checkpoint:** App connects to MongoDB successfully.

Part B — Intermediate (0:40–1:20): Mongoose Schema & Model + Create/Read

---

Step 1: Create the model — models/Product.js

```
// models/Product.js
import mongoose from 'mongoose';

const productSchema = new mongoose.Schema(
  {
    name: { type: String, required: true, trim: true },
    price: { type: Number, required: true, min: 0 },
    category: { type: String, default: 'general' },
    inStock: { type: Boolean, default: true },
  },
  { timestamps: true }
);

export default mongoose.model('Product', productSchema);
```

Step 2: Create routes/products.js with Create and Read routes

```
// routes/products.js
import express from 'express';
import Product from '../models/Product.js';

const router = express.Router();

// CREATE
router.post('/', async (req, res) => {
  try {
    const product = await Product.create(req.body);
    res.status(201).json(product);
  } catch (err) {
    res.status(400).json({ error: err.message });
  }
});

// READ ALL
router.get('/', async (req, res) => {
  try {
    const products = await Product.find();
    res.json(products);
  } catch (err) {
    res.status(500).json({ error: err.message });
```

```
  }
});

// READ ONE
router.get('/:id', async (req, res) => {
  try {
    const product = await Product.findById(req.params.id);
    if (!product) return res.status(404).json({ error: 'Product not found' });
    res.json(product);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

export default router;
```

Step 3: Wire the router into server.js

```
import productsRouter from './routes/products.js';
// ... after app.use(express.json());
app.use('/products', productsRouter);
```

Step 4: Test with curl

```
curl -X POST http://localhost:3000/products \
  -H "Content-Type: application/json" \
  -d '{"name":"Laptop","price":25000,"category":"electronics"}'

curl http://localhost:3000/products
```

✅ **Checkpoint:** You can create and read documents from MongoDB via the API.

Part C — Advanced (1:20–2:00): Update, Delete, Validation & Query Filters

---

Step 1: Add Update and Delete routes

Append to routes/products.js:

```
// UPDATE
router.put('/:id', async (req, res) => {
  try {
    const product = await Product.findByIdAndUpdate(req.params.id, req.body, {
      new: true,
```

```
      runValidators: true,
    });
    if (!product) return res.status(404).json({ error: 'Product not found' });
    res.json(product);
  } catch (err) {
    res.status(400).json({ error: err.message });
  }
});

// DELETE
router.delete('/:id', async (req, res) => {
  try {
    const product = await Product.findByIdAndDelete(req.params.id);
    if (!product) return res.status(404).json({ error: 'Product not found' });
    res.json({ message: 'Product deleted', product });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});
```

Step 2: Add query filtering & sorting to the READ ALL route

Replace the GET / handler:

```
router.get('/', async (req, res) => {
  try {
    const { category, minPrice, maxPrice, sort } = req.query;
    const filter = {};
    if (category) filter.category = category;
    if (minPrice || maxPrice) {
      filter.price = {};
      if (minPrice) filter.price.$gte = Number(minPrice);
      if (maxPrice) filter.price.$lte = Number(maxPrice);
    }

    let query = Product.find(filter);
    if (sort) query = query.sort(sort); // e.g. ?sort=price or ?sort=-price

    const products = await query;
    res.json(products);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});
```

Step 3: Test filters

```
curl "http://localhost:3000/products?category=electronics&minPrice=1000&sort=-price"
```

Step 4: Test update & delete

```
curl -X PUT http://localhost:3000/products/<id> \
  -H "Content-Type: application/json" \
  -d '{"price":22000}'

curl -X DELETE http://localhost:3000/products/<id>
```

(Replace <id> with a real _id from a previous GET response.)

Challenge (optional)

Add a reviews sub-document array to the Product schema (each review has user, rating, comment) and a POST /products/:id/reviews route to add a review.

Step 5: Commit

```
git init
git add .
git commit -m "Week 4: MongoDB + Mongoose full CRUD API with filters"
```

Submission Checklist

- ☐ models/Product.js, routes/products.js, server.js
- ☐ Screenshot of MongoDB Atlas/Compass showing saved documents
- ☐ Postman/curl output for all 5 operations (Create, Read all, Read one, Update, Delete)